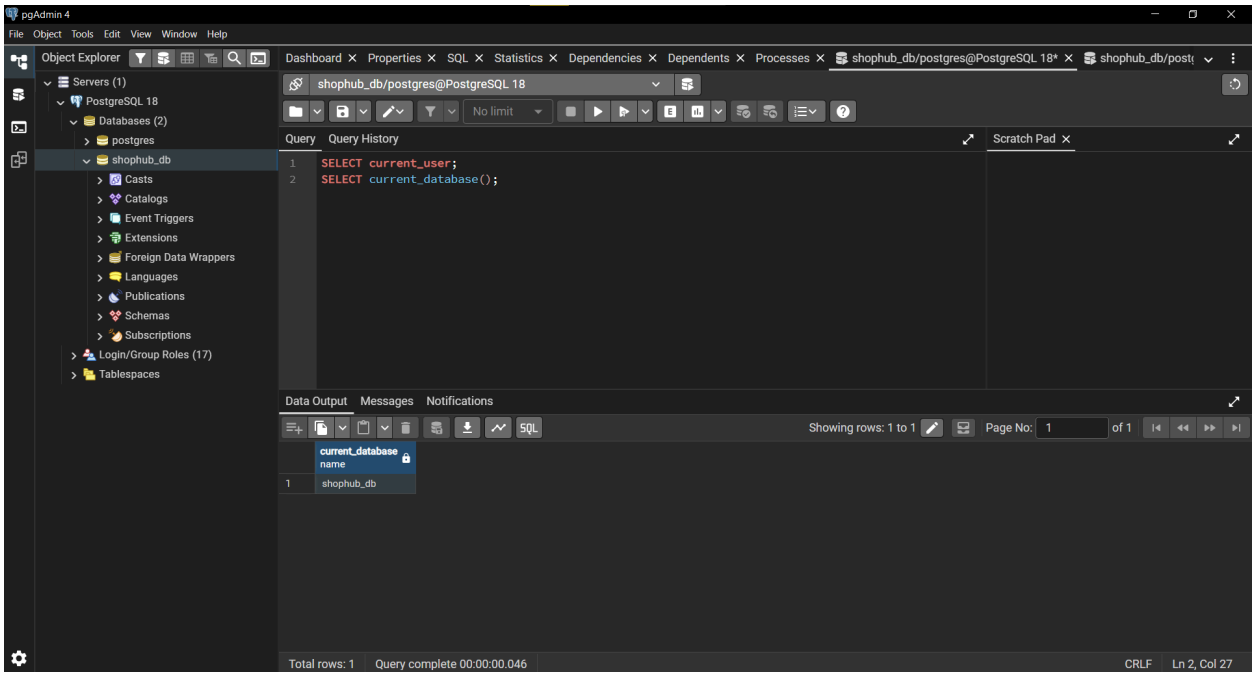
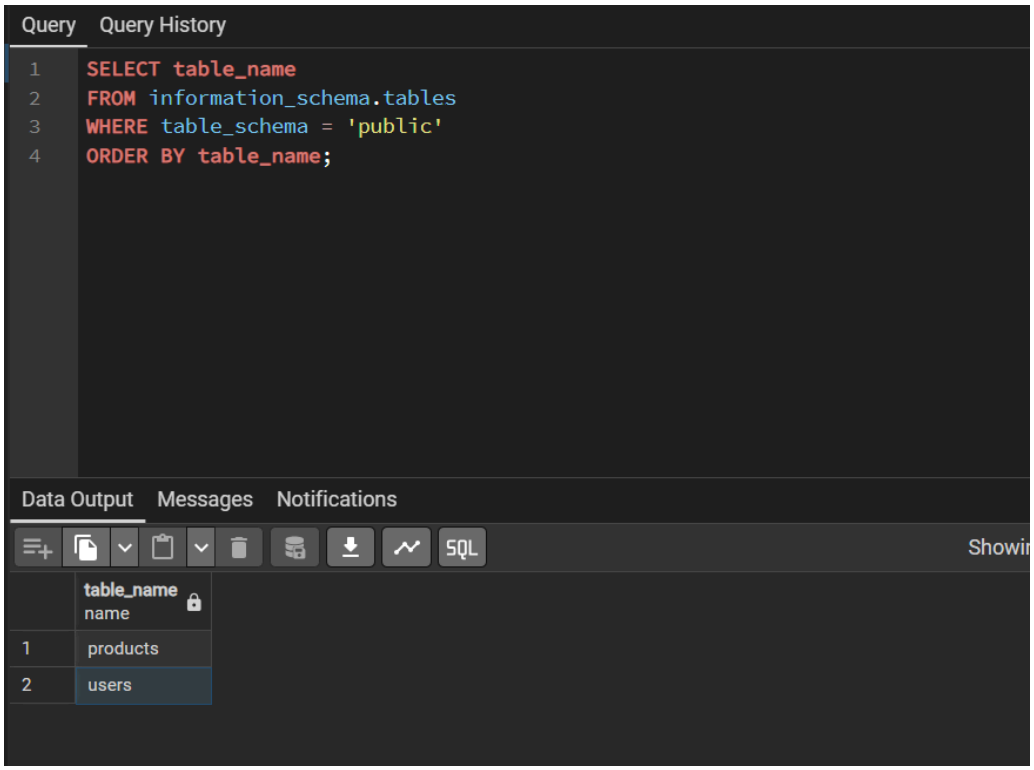
1. PostgreSQL database connection



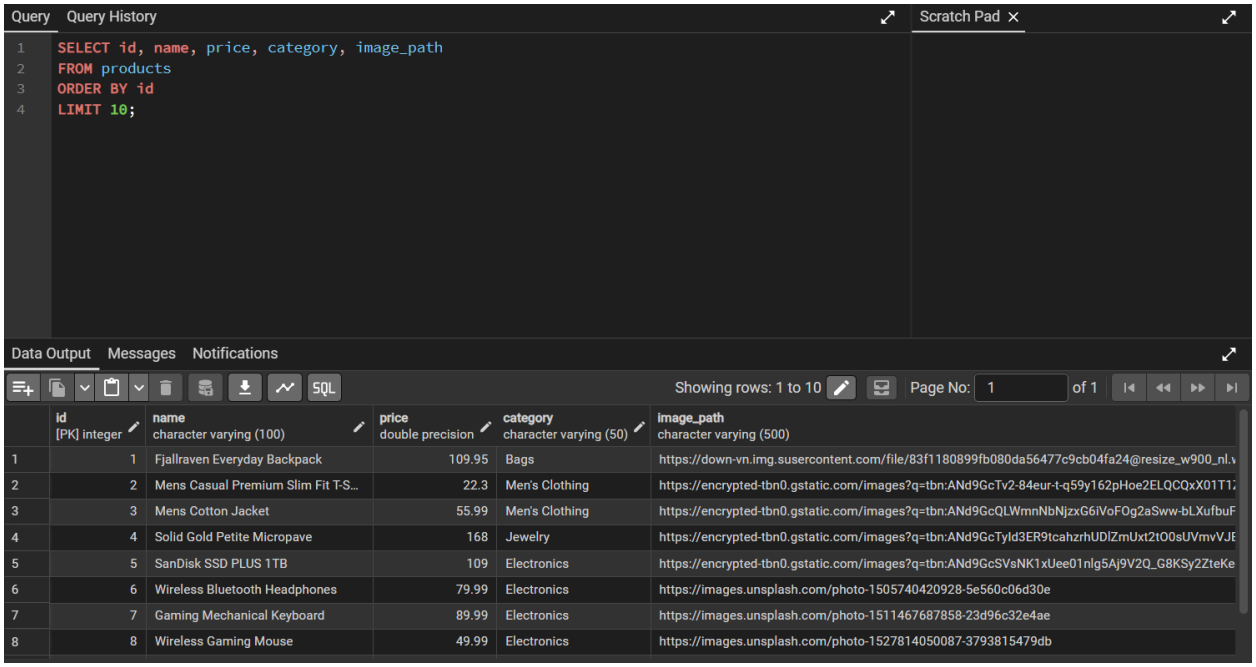
The FastAPI project is using the PostgreSQL database shophub_db with the postgres user.

2. ProductDB and UserDB tables exist



The products and users tables were created in PostgreSQL using SQLAlchemy models.

3. Product data imported/stored in PostgreSQL



Query | Query History | Scratch Pad

```
SELECT id, name, price, category, image_path
FROM products
ORDER BY id
LIMIT 10;
```

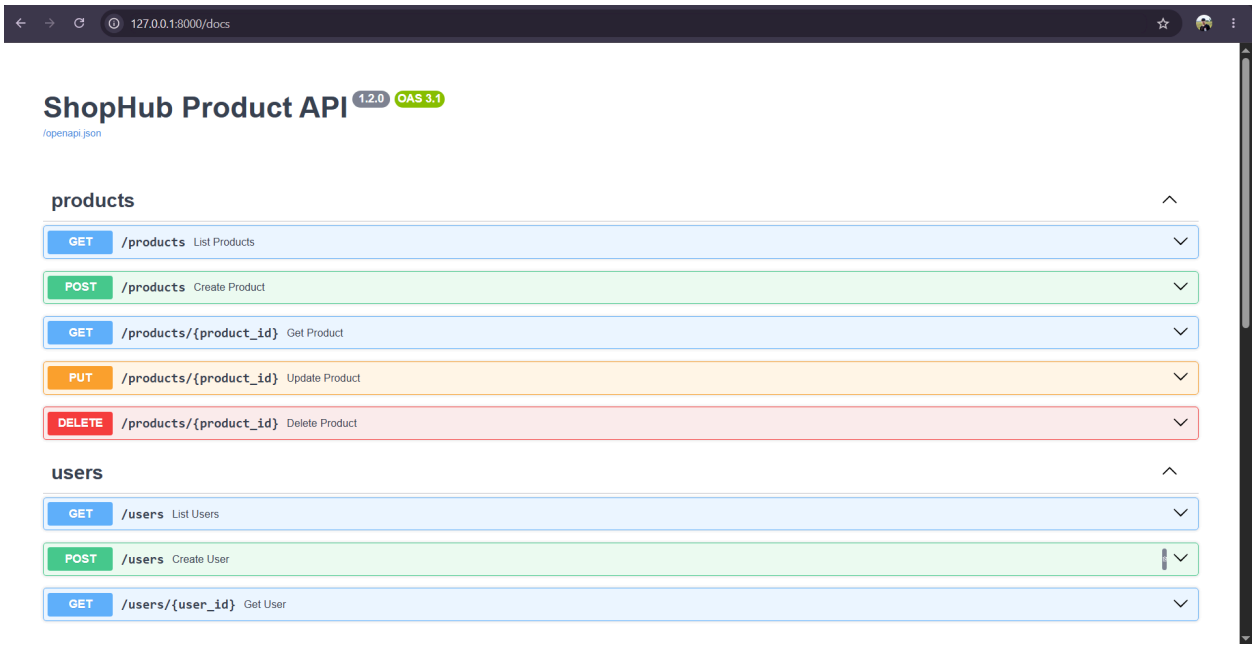
Data Output | Messages | Notifications

Showing rows: 1 to 10 | Page No: 1 of 1

	id [PK] integer	name character varying (100)	price double precision	category character varying (50)	image_path character varying (500)
1	1	Fjallraven Everyday Backpack	109.95	Bags	https://down-vn.img.susercontent.com/file/83f1180899fb080da56477c9cb04fa24@resize_w900_nl.v
2	2	Mens Casual Premium Slim Fit T-S...	22.3	Men's Clothing	https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcTv2-84eur-t-q59y162pHoe2ELQCQxX01T1
3	3	Mens Cotton Jacket	55.99	Men's Clothing	https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcQLWmnNbNjzxG6iVoFOg2aSww-bLXufbuF
4	4	Solid Gold Petite Micropave	168	Jewelry	https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcTyld3ER9tcahzrhUDlZmUxt2tO0sUVmvVJE
5	5	SanDisk SSD PLUS 1TB	109	Electronics	https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcSVsNK1xUee01nlg5Aj9V2Q_G8KSy2ZteKe
6	6	Wireless Bluetooth Headphones	79.99	Electronics	https://images.unsplash.com/photo-1505740420928-5e560c06d30e
7	7	Gaming Mechanical Keyboard	89.99	Electronics	https://images.unsplash.com/photo-1511467687858-23d96c32e4ae
8	8	Wireless Gaming Mouse	49.99	Electronics	https://images.unsplash.com/photo-1527814050087-3793815479db

Existing product data is now stored in the PostgreSQL products table instead of only in products.json.

4. Swagger shows Product CRUD endpoints



127.0.0.1:8000/docs

ShopHub Product API 1.2.0 OAS 3.1

/openapi.json

products

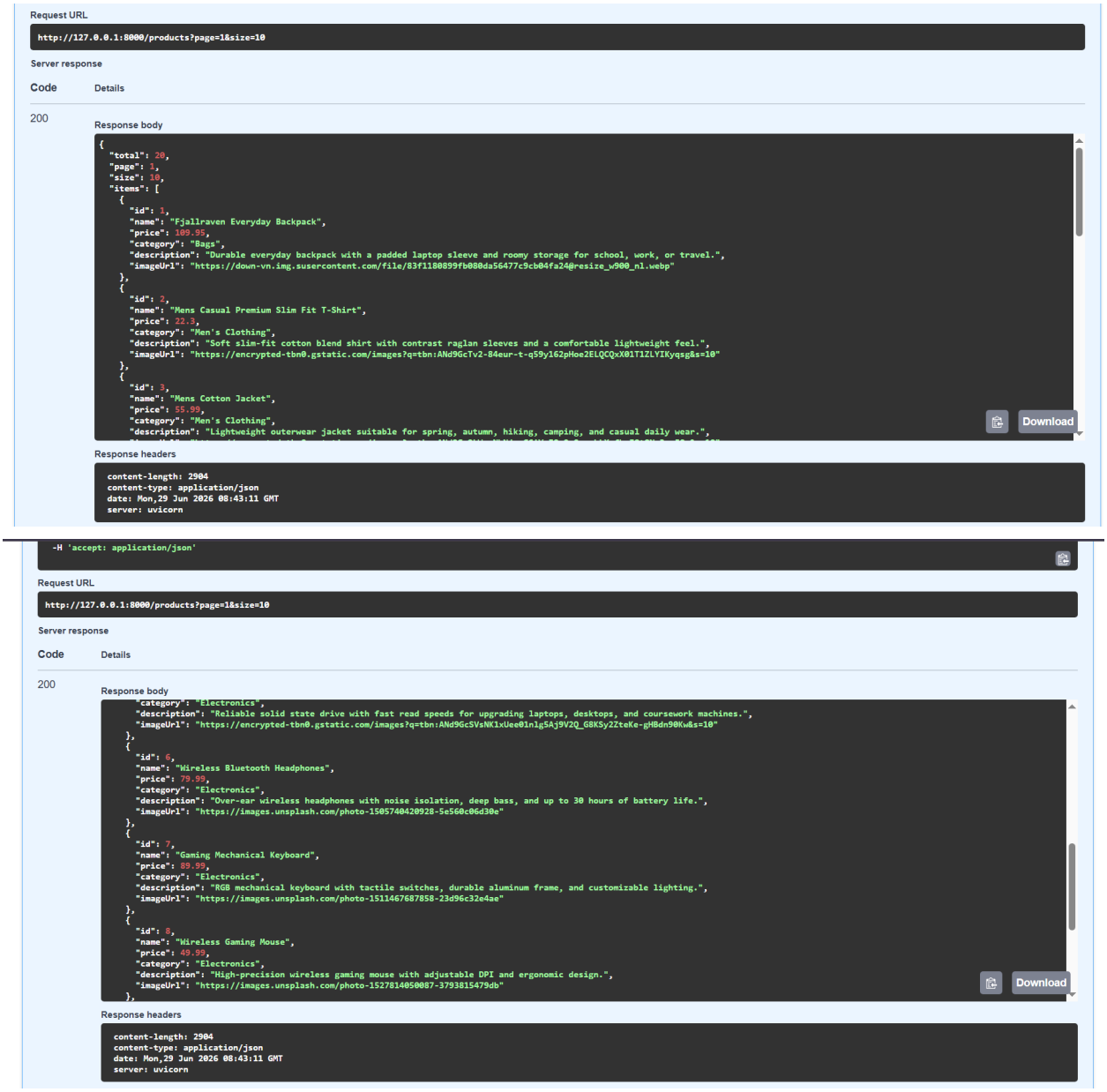
- GET /products List Products
- POST /products Create Product
- GET /products/{product_id} Get Product
- PUT /products/{product_id} Update Product
- DELETE /products/{product_id} Delete Product

users

- GET /users List Users
- POST /users Create User
- GET /users/{user_id} Get User

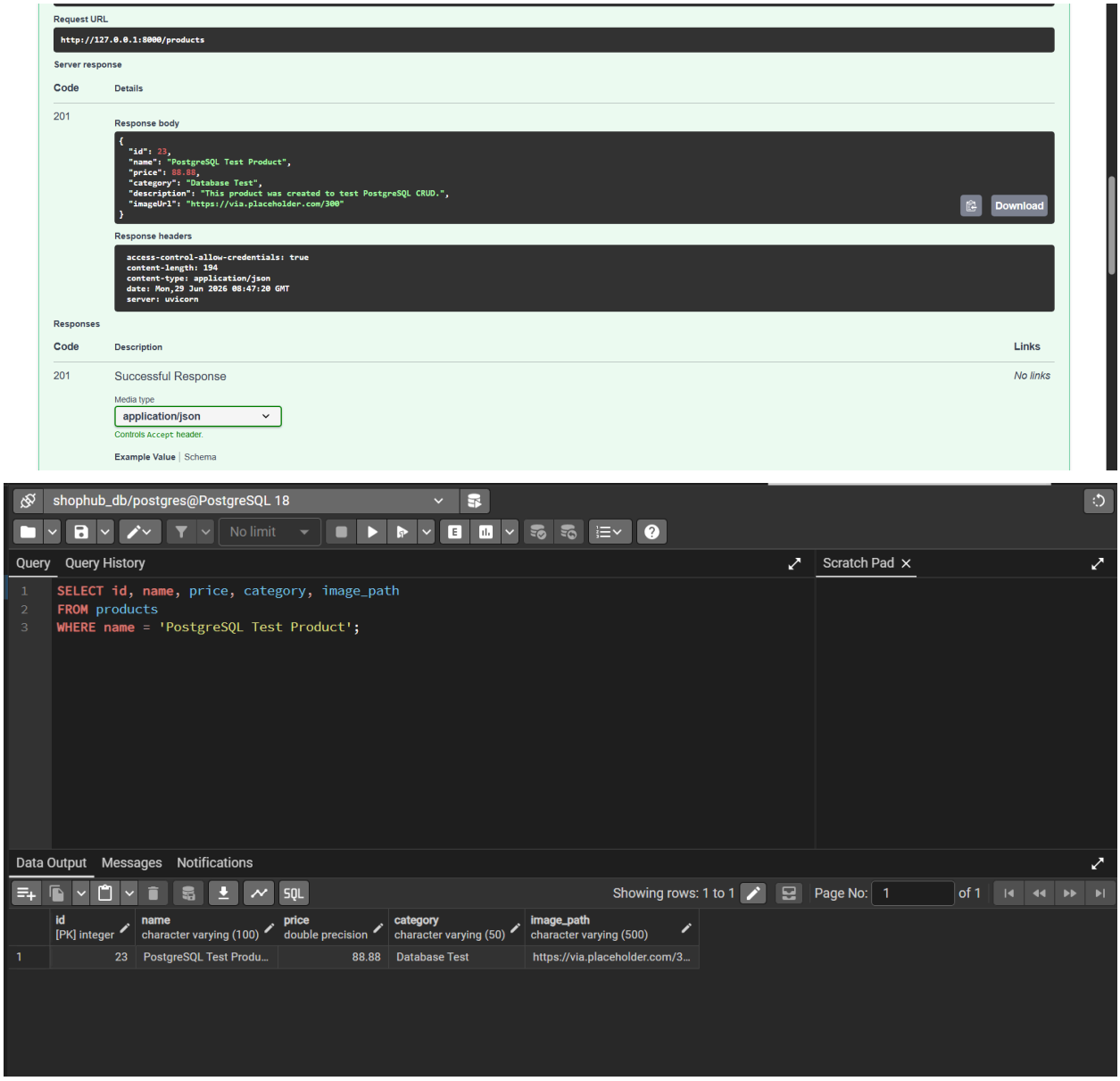
Swagger UI shows the Product CRUD endpoints implemented for the PostgreSQL backend.

5. GET /products reads from PostgreSQL



GET /products retrieves paginated product data from PostgreSQL.

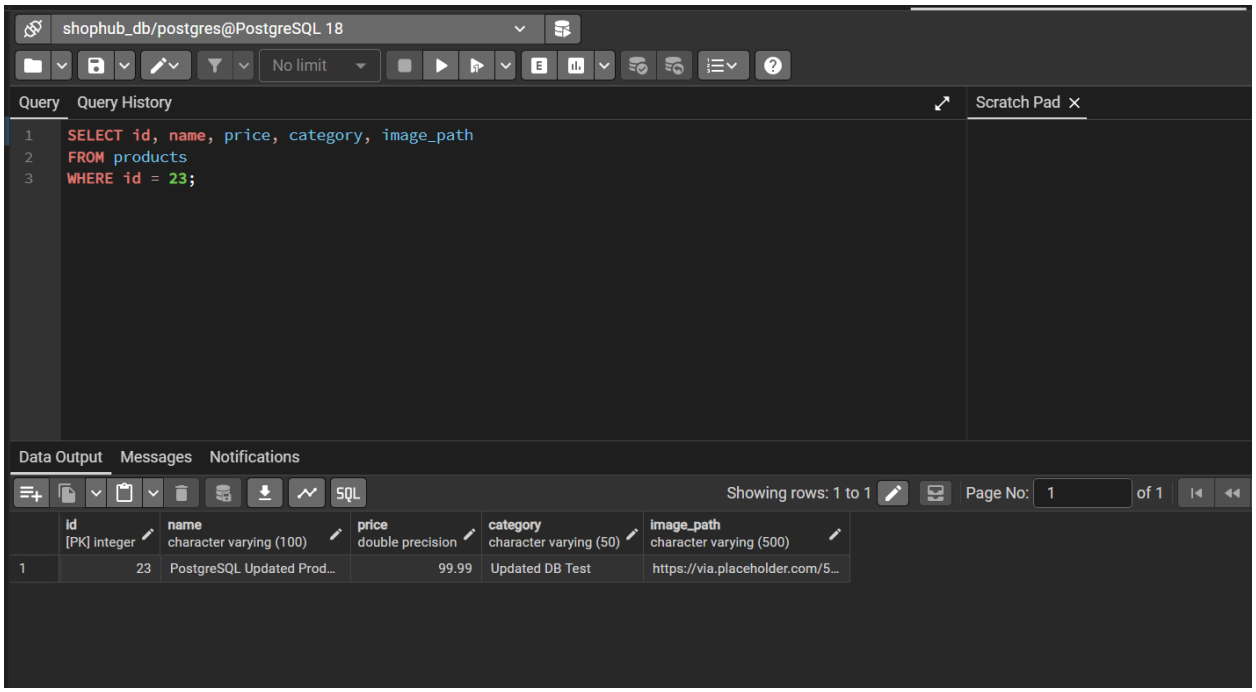
6. POST /products creates data in PostgreSQL



The product created from Swagger is persisted in the PostgreSQL products table.

7. PUT /products/{id} updates PostgreSQL data

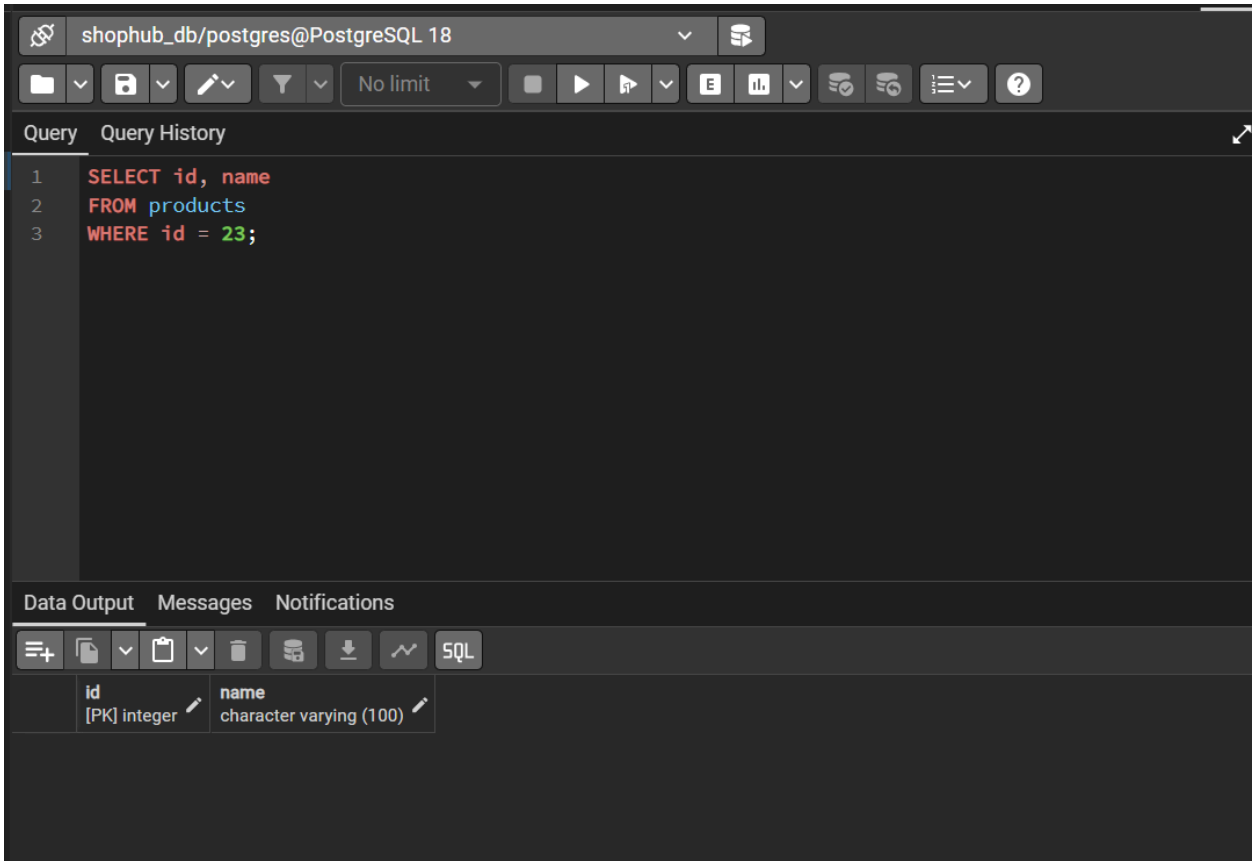
PUT /products/{id} updates product information in PostgreSQL.



The updated product values are saved in the PostgreSQL database.

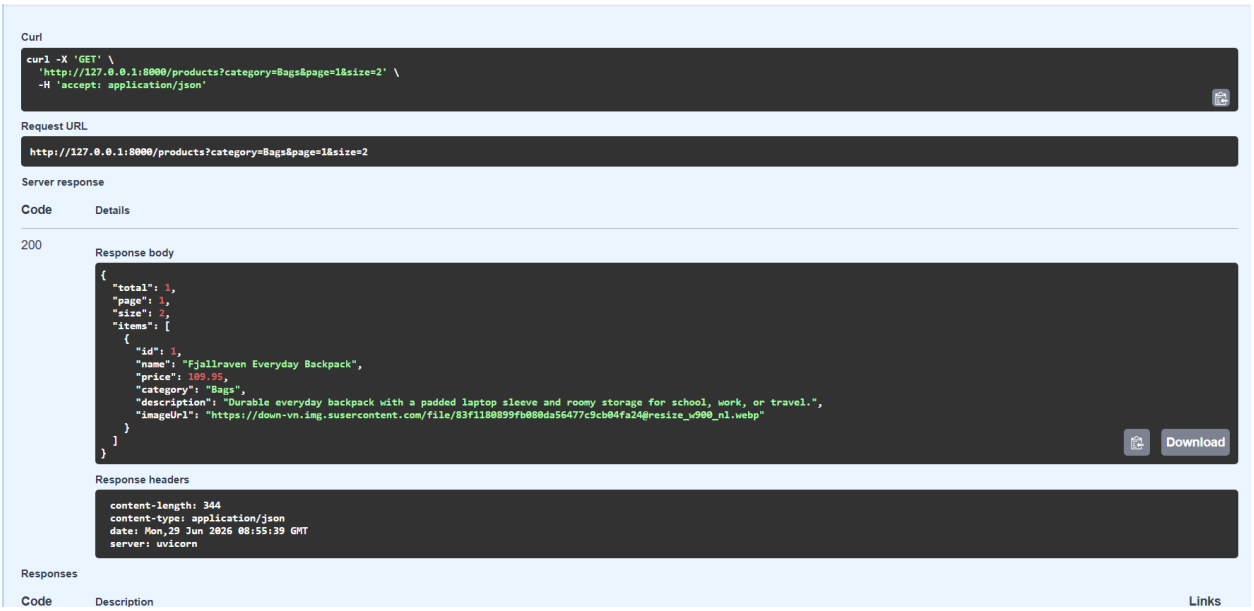
8. DELETE /products/{id}

DELETE /products/{id} removes a product from PostgreSQL.



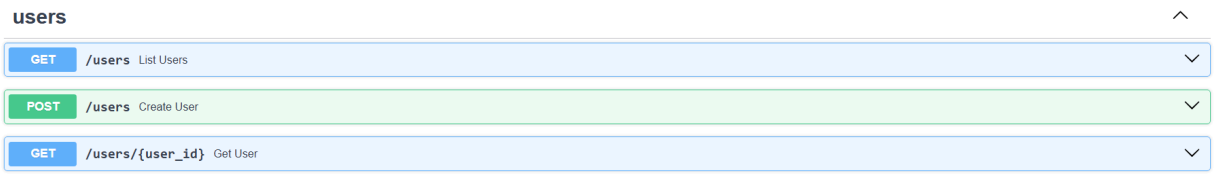
The deleted product no longer exists in the PostgreSQL products table.

9. Filter and pagination homework



GET /products supports SQLAlchemy filtering and pagination using category, page, and size query parameters.

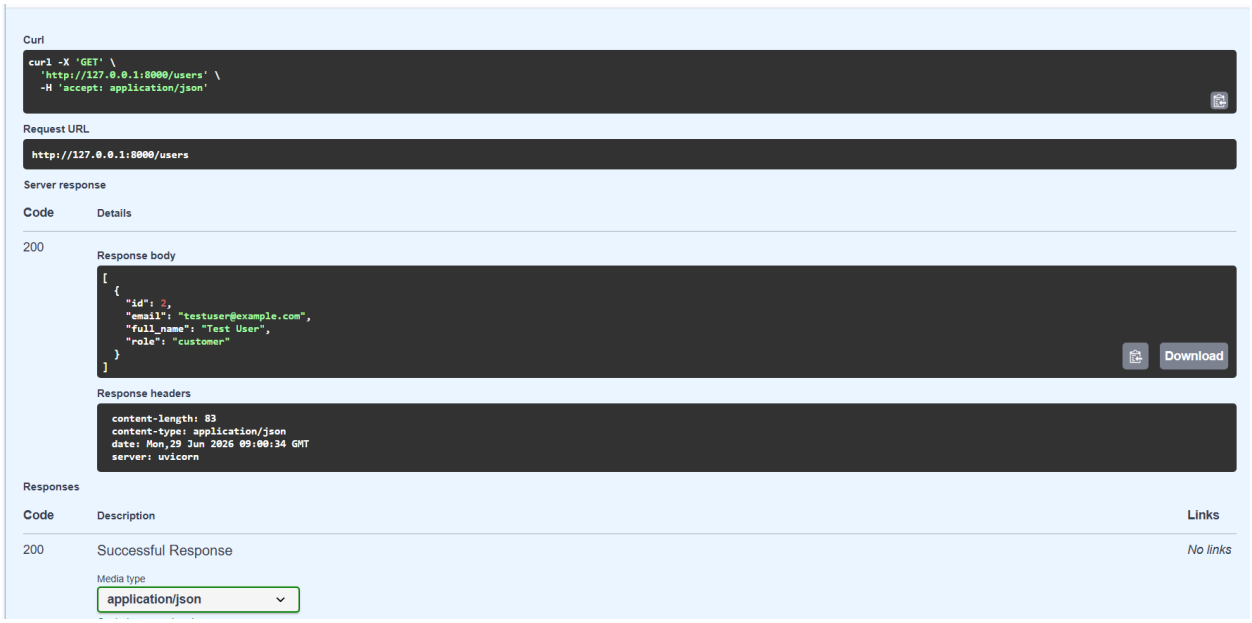
10. User endpoints



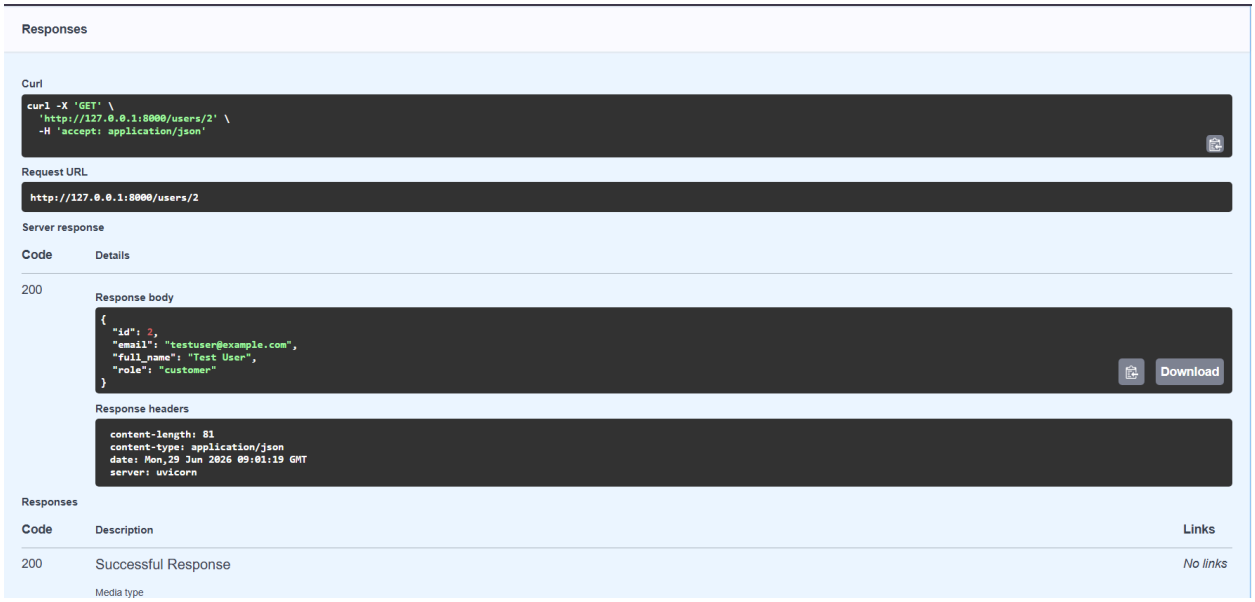
Swagger UI shows basic User endpoints for future authentication features.

POST /users creates a user while hiding sensitive password_hash from the public response.

11. GET /users and GET /users/{id}

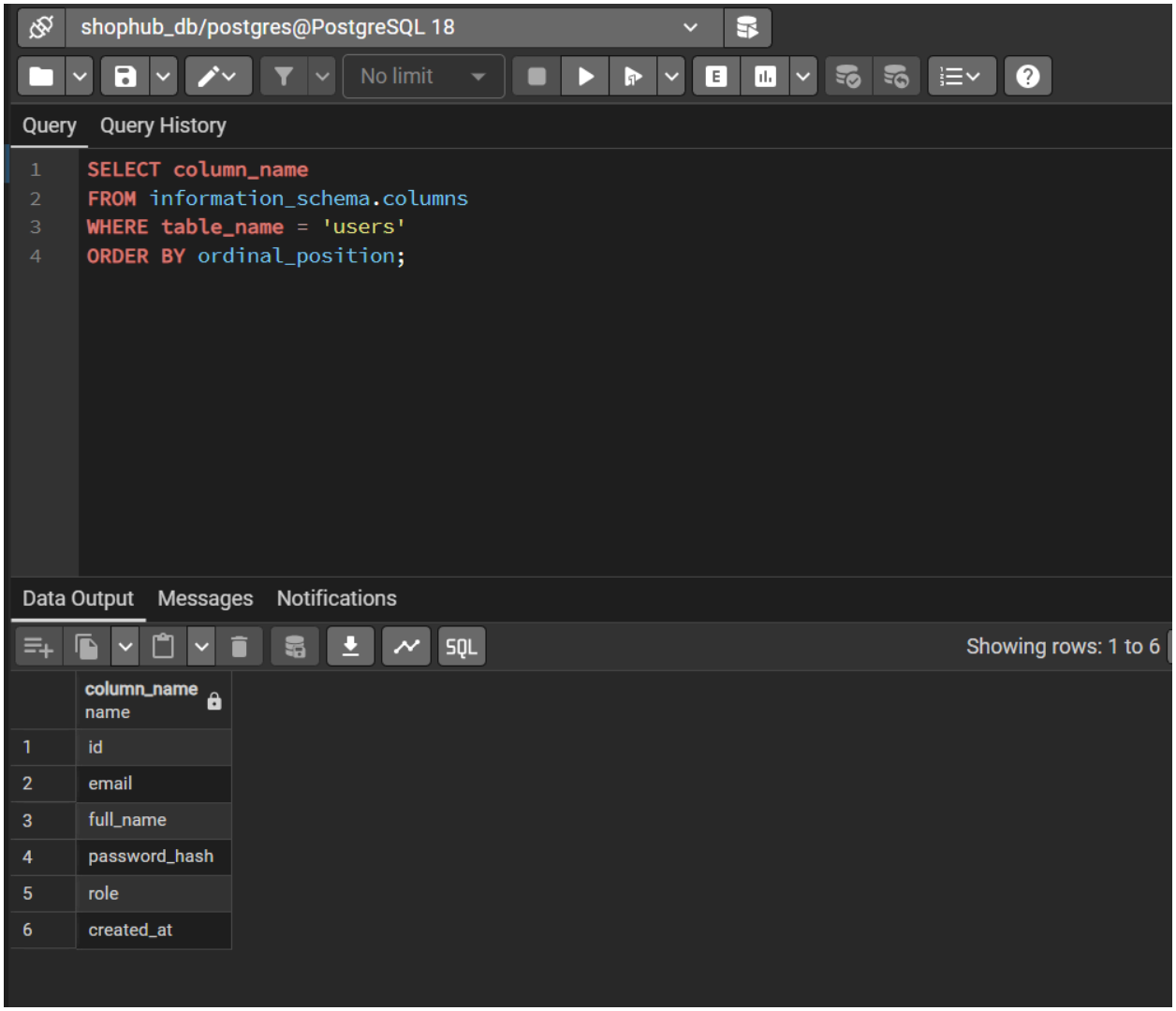


GET /users returns public user data using UserRead and does not expose password_hash.



GET /users/{id} retrieves safe user details from PostgreSQL.

12. Prove password_hash exists internally but hidden publicly



The users table stores password_hash internally, but API response models hide it from public output.

PostgreSQL is better than JSON files for multi-user applications because it supports structured queries, persistent tables, safer concurrent access, indexing, and relationships between data. A JSON file is simple for learning, but it is not suitable when many users or requests need to read and update data at the same time.

Separating database models and Pydantic schemas improves security and maintainability. Database models can store internal fields such as image_path, created_at, updated_at, or password_hash, while public response schemas only expose safe fields such as id, name, price, email, and role. This prevents sensitive data from being accidentally returned to the frontend and keeps the API easier to change later.